**МИНОБРНАУКИ РОССИИ**
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**
**Кафедра МОЭВМ**

ОТЧЕТ

**по лабораторной работе №2**

**по дисциплине «Интеллектуальные методы анализа данных»**

Тема: Реализация метода классификации данных с использованием природных временных рядов

Студентка гр. 3381 _____ Уфимцева А.А.

Преподаватель _____ Мандрикова Б.С.

Санкт-Петербург

2026

**Цель работы.**

Построение классификатора данных геофизического мониторинга на примере наблюдений нейтронных мониторов.

**Задание.**

Вариант 17 Москва (MOSC) 1-9 ноября 2018

1. Подготовить данные. Скачать данные из источника [https://www.nmdb.eu/nest/] за периоды и станции, указанные по вариантам таблицах. Использовать сданные с одноминутным временным разрешением. При наличии пропусков в данных заполнить их медианным значением (функция median в MatLab).
2. Сформировать векторы-историй. Временной ряд разбить на последовательные блоки (векторы) длиной m, каждый из которых должен содержать m прошлых значений. Последний вектор в рядя будет «тестовой» точкой для поиска соседей. Величину m выбрать равной 180 отсчетов.
3. Найти ближайших соседей. Для «тестового» вектора вычислить расстояние (метрику) до всех остальных векторов-историй (использовать 3 различные метрики). Число соседей (k) определить равным 2-4.
4. Использовать соседей для прогноза. На основе найденных ближайших соседей. Построить предсказания для будущих значений ряда. При нахождении нескольких схожих отрезков, нужно взять простое среднее следующих за ними значений.
5. Сделать соответствующие выводы, оформить отчет по работе.

**Выполнение работы.**

Загрузка данных: Данные были получены с сайта NMDB (https://www.nmdb.eu/nest/) за заданный период. Файл данных сохранен в формате HTML. Для загрузки и парсинга данных использовался язык Python с библиотеками pandas, numpy и matplotlib. Также было принято решение предсказывать следующие 60 значений за «тестовым» вектором, поэтому также были скачаны эти значения в отдельный файл.

Обработка данных: В обоих файлах, аналогично предыдущей работе были найдены пропуски, выбросы и поменяны на медианное значение. (В файле с фактическими данными  60 значений за «тестовым» вектором пропусков и выбросов не было обнгаружено).

Далее создаем вектора истории: Берем по порядку все вектора длины 180, не пересекающиеся с «тестовым» вектором. Для каждого из этих векторов берем 60 следующих для них значений, чтобы в последствие строить предсказания.

Поиск ближайших соседей: Для поиска ближайших соседей было выбранно 3 метрики из лекции: Евклидова, Манхэттенская и Чебышева. Для каждого вектора-истории была посчитана эта метрика с «тестовым» вектором. Число соседей было определено как 3, поэтому дальше для каждой метрики были взяты 3 вектора с наименьшими расстояниями по данным метрикам.

Полученные результаты по метрикам:

метрика: Евклидова
 сосед 1: индекс=12421, расстояние=57.349267
 сосед 2: индекс=7088, расстояние=57.975280
 сосед 3: индекс=7178, расстояние=58.052953

метрика: Манхэттенская
 сосед 1: индекс=7088, расстояние=611.290000
 сосед 2: индекс=7178, расстояние=612.190000
 сосед 3: индекс=11418, расстояние=618.220000

метрика: Чебышева
 сосед 1: индекс=11450, расстояние=10.030000
 сосед 2: индекс=12540, расстояние=10.040000

сосед 3: индекс=1398, расстояние=10.140000

Прогноз по соседям: Для ближайших соседей по каждой метрике было найдено среднее арифметическое 60 значений, следующих за векторами-соседями. Это и будет предсказанием по каждой из метрик. Далее на рисунках 1-3 представлены графики предсказанных значений для каждой метрики и фактическое значение.

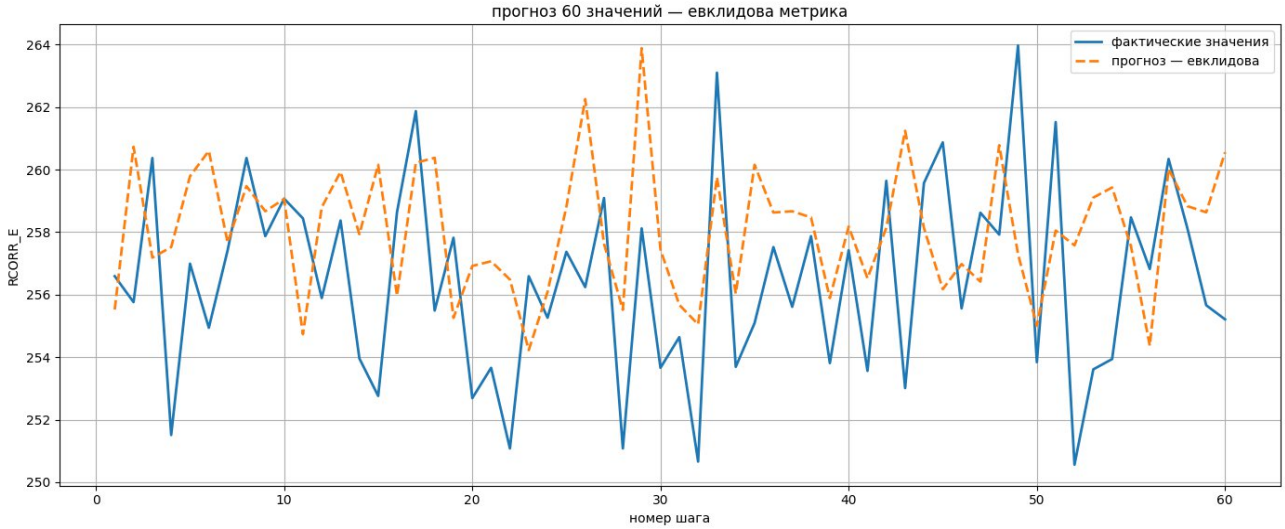



Рис. 1 — Прогноз по евклидовой метрике

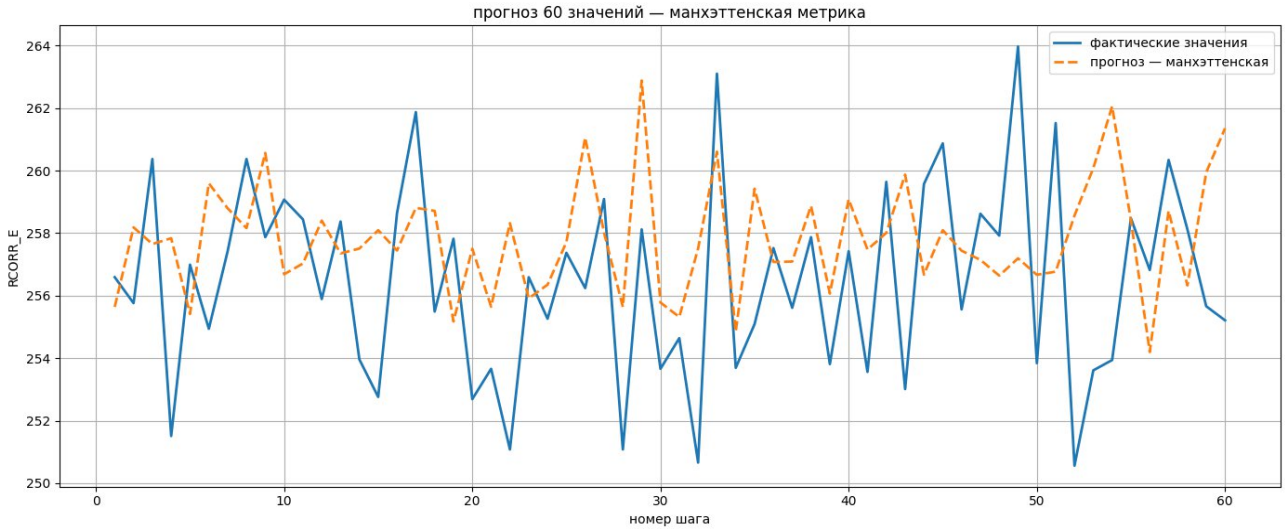



Рис. 2 — Прогноз по манхэттенской метрике

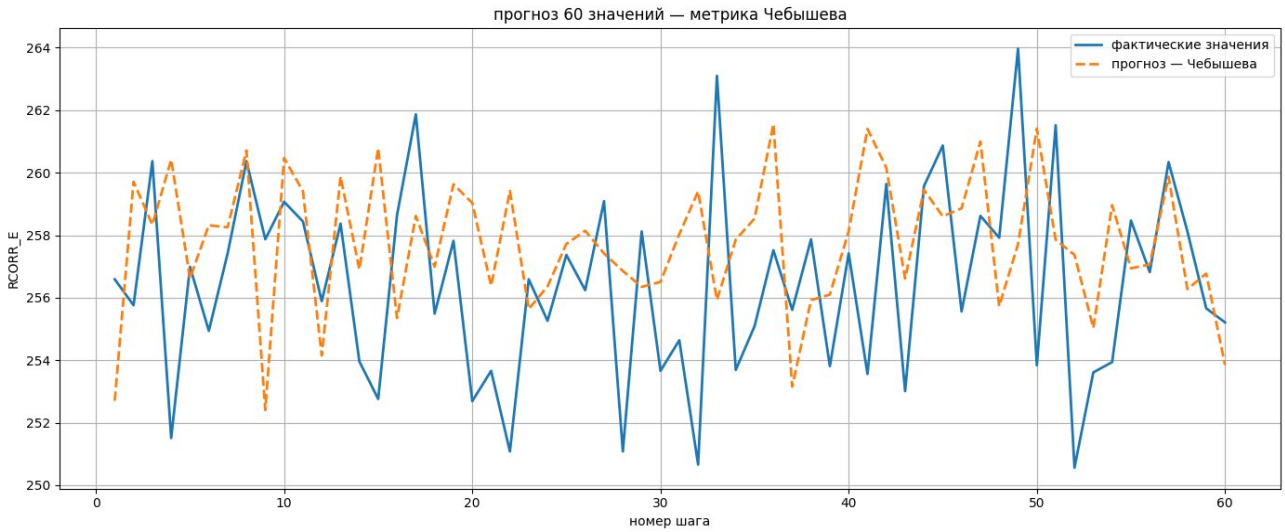



Рис. 3 — Прогноз по метрике Чебышева

Прогнозы, построенные по трём метрикам, визуально различаются. Это связано с тем, что евклидова, манхэттенская и чебышевская метрики по-разному определяют «похожесть» векторов-историй: Евклидова ориентируется на суммарное отклонение с усиленным штрафом за большие разности, манхэттенская — на равномерное сходство по всем координатам, Чебышева — на совпадение по наихудшей координате. Соответственно, и наборы ближайших соседей, и их «будущие» значения оказываются разными. Также видим, что графики прогнозируемых значений не особо похожи на фактические значения, что может быть из-за непохожести данных на «тестовый вектор», неправильных значений k или m или выбранных метрик.

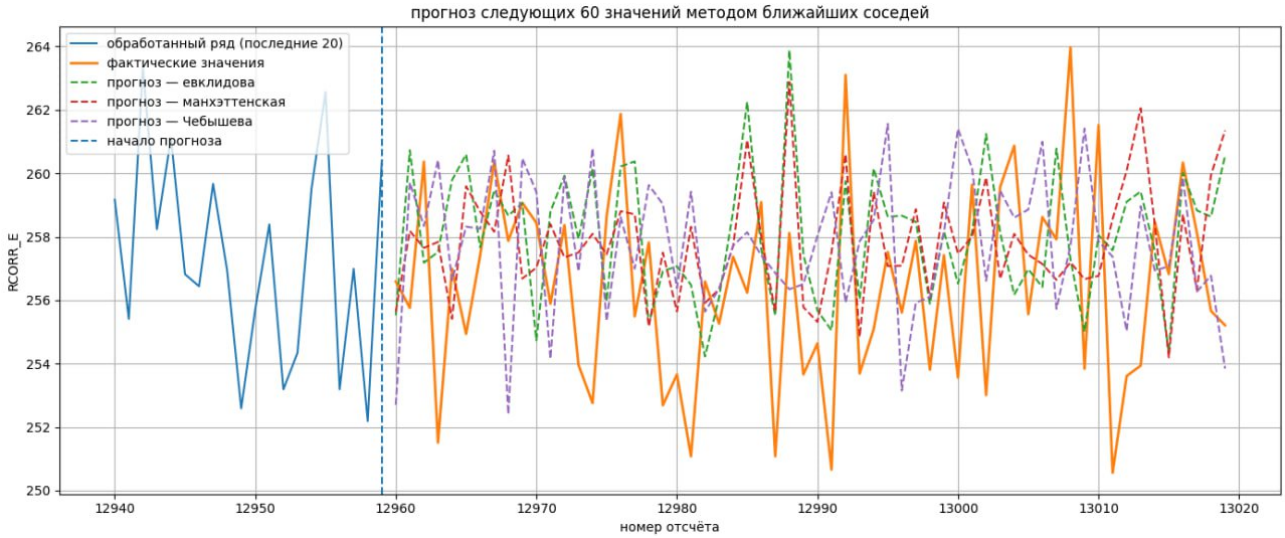



Рис. 4 — Все предсказанные ряды

Также для всех трех предсказанных значений ряда была почитана метрика MAE.

MAE:

евклидова: 3.119833

манхэттенская: 3.054556

чебышева: 3.121667

MAE считает среднюю абсолютную ошибку — то есть насколько в среднем прогноз отличается от факта. Как можно заметить, значения примерно равны. Так как разброс значений в ряду ~14 единиц, а средняя ошибка на всем ряду ~3 единицы, что составляет почти четверть размаха, то можно сказать, что предугаданный ряд не похож на фактический. Это видно и по графикам. Делаем вывод, что данный метод предсказания временного ряда работает плохо.

**Выводы.**

В ходе работы был построен прогноз временного ряда нейтронного монитора методом ближайших соседей на векторах-историях. В качестве меры ошибки использовалась MAE (средняя абсолютная ошибка) — она показывает, насколько в среднем прогноз отличается от факта. Как можно заметить, значения MAE для всех трёх метрик примерно равны и лежат в узком диапазоне 3.05–3.12. Это означает, что выбор метрики расстояния при поиске соседей практически не влияет на итоговое качество прогноза в данной задаче: все три метрики дают сопоставимо плохой результат. Причина в самом методе: прогноз строится как простое среднее следующих значений k = 3 соседей. Тестовый вектор оказался «пограничным» — ни одна из метрик не нашла соседей, действительно похожих на него по всей 180-точечной истории; три метрики отобрали разные тройки соседей, и их прогнозы заметно разошлись между собой, что говорит о ненадёжности выбора. Таким образом, данный метод предсказания временного ряда на 60-минутном горизонте работает плохо.

## ПРИЛОЖЕНИЕ А. ИСХОДНЫЙ КОД ПРОГРАММЫ.

Название файла: imad2.py

```
import pandas as pd
```

```
import numpy as np
import matplotlib.pyplot as plt
import os
import re
from io import StringIO

#обработка файла фактических значений
filename2 = 'draw_graph.php2.csv'

if not os.path.exists(filename2):
    print(f"файл {filename2} не найден.")
    exit()

try:
    with open(filename2, 'r', encoding='utf-8') as f:
        html_content2 = f.read()

    lines2 = html_content2.split('\n')
    data_lines2 = []

    for line in lines2:
        line = line.strip()

        if re.match(r'^\d{4}-\d{2}-\d{2}', line):
            data_lines2.append(line)

    if not data_lines2:
        print("ошибка: в первом файле не найдены строки с данными.")
        exit()

    clean_data_str2 = '\n'.join(data_lines2)

    df2 = pd.read_csv(
        StringIO(clean_data_str2),
        sep=';',
        header=None,
        names=['Timestamp', 'Value']
    )

    print("первые 5 строк файла с фактическими данными:")
    print(df2.head())
    print()

    time_data2 = pd.to_datetime(
        df2['Timestamp'],
        errors='coerce'
    )

    raw_data2 = pd.to_numeric(
        df2['Value'].astype(str).str.replace(',', '.'),
        errors='coerce'
    )

    if raw_data2.isna().all():
        print("ошибка: не удалось преобразовать значения в числа в файле.")
        exit()

    total_records2 = len(raw_data2)
    print(f"общее количество записей в файле: {total_records2}")
    print()

except Exception as e:
    print(f"ошибка при чтении или парсинге файла: {e}")
```

```
    exit()

#обработка данных файла

#подсчет пропусков
missing_mask2 = raw_data2.isna()
count_missing2 = missing_mask2.sum()
share_missing2 = count_missing2 / len(raw_data2) * 100

print(f"количество пропусков: {count_missing2}")
print(f"доля пропусков: {share_missing2:.2f}%")

#считаем статистику по данным без пропусков
clean_data_for_stats2 = raw_data2.dropna()

if len(clean_data_for_stats2) == 0:
    print("ошибка: не осталось данных для анализа.")
    exit()

mean_val2 = clean_data_for_stats2.mean()
std_val2 = clean_data_for_stats2.std()

#границы: (среднее - 3*ско, среднее + 3*ско)
lower_bound2 = mean_val2 - 3 * std_val2
upper_bound2 = mean_val2 + 3 * std_val2

#маска выбросов
outlier_mask2 = (raw_data2 < lower_bound2) | (raw_data2 > upper_bound2)
count_outliers2 = outlier_mask2.sum()
share_outliers2 = count_outliers2 / len(raw_data2) * 100

print(f"границы нормы (3 сигмы): [{lower_bound2:.2f}, {upper_bound2:.2f}]")
print(f"количество выбросов: {count_outliers2}")
print(f"доля выбросов: {share_outliers2:.2f}%")

#вычисление медианы по чистым данным
clean_data2 = raw_data2[~missing_mask2 & ~outlier_mask2]
median_value2 = clean_data2.median()

print(f"медианное значение (без пропусков и выбросов): {median_value2:.4f}")

#замена пропусков и выбросов медианой
processed_data2 = raw_data2.copy()
bad_points_mask2 = missing_mask2 | outlier_mask2
processed_data2[bad_points_mask2] = median_value2

print("замена выполнена")
print()


#проверка фактических 60 значений
actual_60 = processed_data2.values.astype(float)
horizon = 60

print(f"количество значений для проверки: {len(actual_60)}")
if len(actual_60) != horizon:
    print(
        f"ошибка: найдено {len(actual_60)} значений, "
        f"а должно быть {horizon}."
    )
    exit()
print(f"проверка пройдена: найдено ровно {horizon} значений.")
print()
```

```
#обработка файла с историческими данными
filename = 'draw_graph.php.html'

if not os.path.exists(filename):
    print(f"Ошибка: Файл {filename} не найден.")
    exit()

try:
    with open(filename, 'r', encoding='utf-8') as f:
        html_content = f.read()

    match = re.search(
        r'<pre><code>(.*?)</code></pre>',
        html_content,
        re.DOTALL
    )

    if not match:
        print("ошибка: не найден блок с данными.")
        exit()

    raw_text_data = match.group(1)

    lines = raw_text_data.split('\n')
    data_lines = []

    for line in lines:
        line = line.strip()

        if line and not line.startswith('#'):
            data_lines.append(line)

    start_idx = 0

    for i, line in enumerate(data_lines):
        if re.match(r'^\d{4}-\d{2}-\d{2}', line):
            start_idx = i
            break

    clean_data_str = '\n'.join(data_lines[start_idx:])

    df = pd.read_csv(
        StringIO(clean_data_str),
        sep=';',
        header=None,
        names=['Timestamp', 'Value']
    )

    print("первые 5 строк исторических данных:")
    print(df.head())
    print()

    time_data = pd.to_datetime(
        df['Timestamp'],
        errors='coerce'
    )

    raw_data = pd.to_numeric(
        df['Value'].astype(str).str.replace(',', '.'),
        errors='coerce'
    )
```

```
    if raw_data.isna().all():
        print("ошибка: не удалось преобразовать значения в числа.")
        exit()

    total_records = len(raw_data)

    print(f"общее количество исторических записей: {total_records}")
    print()

except Exception as e:
    print(f"ошибка при чтении или парсинге исторического файла: {e}")
    exit()

#обработка исторических данных
#подсчет пропусков
missing_mask = raw_data.isna()
count_missing = missing_mask.sum()
share_missing = count_missing / len(raw_data) * 100

print(f"количество пропусков: {count_missing}")
print(f"доля пропусков: {share_missing:.2f}%")

#считаем статистику по данным без пропусков
clean_data_for_stats = raw_data.dropna()

if len(clean_data_for_stats) == 0:
    print("ошибка: не осталось данных для анализа.")
    exit()

mean_val = clean_data_for_stats.mean()
std_val = clean_data_for_stats.std()

#границы: (среднее - 3*ско, среднее + 3*ско)
lower_bound = mean_val - 3 * std_val
upper_bound = mean_val + 3 * std_val

#маска выбросов
outlier_mask = (raw_data < lower_bound) | (raw_data > upper_bound)
count_outliers = outlier_mask.sum()
share_outliers = count_outliers / len(raw_data) * 100

print(f"границы нормы (3 сигмы): [{lower_bound:.2f}, {upper_bound:.2f}]")
print(f"количество выбросов: {count_outliers}")
print(f"доля выбросов: {share_outliers:.2f}%")

#вычисление медианы по "чистым" данным
clean_data = raw_data[~missing_mask & ~outlier_mask]
median_value = clean_data.median()

print(f"медианное значение (без пропусков и выбросов): {median_value:.4f}")

#замена пропусков и выбросов медианой
processed_data = raw_data.copy()
bad_points_mask = missing_mask | outlier_mask
processed_data[bad_points_mask] = median_value

print("замена выполнена.")
print()

#формирование векторов-историй
m = 180
```

```
#количество прогнозируемых будущих значений
horizon = 60

#берём обработанный исторический ряд
series = processed_data.values.astype(float)

N = len(series)

print(f"длина обработанного исторического ряда: {N}")
print(f"длина вектора-истории m = {m}")
print(f"количество прогнозируемых значений = {horizon}")
print()

#тестовый вектор
#последние 180 значений исторического файла
test_vector = series[-m:]

print(f"размер тестового вектора: {len(test_vector)}")
print()

#создание векторов-историй
vectors = []
targets = []

#граница тестового вектора
test_start = N - m

#формируем только вектора, которые полностью находятся до тестового вектора
for i in range(test_start - m + 1):

    #вектор-история длиной 180 значений
    history_vector = series[i:i + m]

    #60 значений, следующих после вектора-истории
    future_vector = series[i + m:i + m + horizon]

    vectors.append(history_vector)
    targets.append(future_vector)


vectors = np.array(vectors)
targets = np.array(targets)

print(f"количество сформированных векторов-историй: {len(vectors)}")
print(f"размер матрицы векторов: {vectors.shape}")
print(f"размер матрицы будущих значений: {targets.shape}")
print()

#поиск ближайших соседей
k = 3

#евклидово расстояние
dist_euclid = np.array([
    np.sqrt(np.sum((test_vector - v) ** 2))
    for v in vectors
])
idx_euclid = np.argsort(dist_euclid)[:k]

#манхэттенское расстояние
dist_manhattan = np.array([
    np.sum(np.abs(test_vector - v))
    for v in vectors
])
```

```
idx_manhattan = np.argsort(dist_manhattan)[:k]

#расстояние Чебышева
dist_chebyshev = np.array([
    np.max(np.abs(test_vector - v))
    for v in vectors
])
idx_chebyshev = np.argsort(dist_chebyshev)[:k]

#вывод найденных ближайших соседей
for name, idx, dists in [
    ("Евклидова", idx_euclid, dist_euclid),
    ("Манхэттенская", idx_manhattan, dist_manhattan),
    ("Чебышева", idx_chebyshev, dist_chebyshev),
]:

    print(f"\n метрика: {name} ")
    for rank, i in enumerate(idx, 1):

        print(
            f"  сосед {rank}: индекс={i}, "
            f"расстояние={dists[i]:.6f}"
        )

#предсказание следующих 60 значений
#берём среднее соответствующих значений у 3 найденных соседей

pred_euclid = np.mean(
    targets[idx_euclid],
    axis=0
)

pred_manhattan = np.mean(
    targets[idx_manhattan],
    axis=0
)

pred_chebyshev = np.mean(
    targets[idx_chebyshev],
    axis=0
)

#график прогноза — евклидова метрика
plt.figure(figsize=(14, 6))

plt.plot(
    range(1, horizon + 1),
    actual_60,
    label='фактические значения',
    linewidth=2
)

plt.plot(
    range(1, horizon + 1),
    pred_euclid,
    label='прогноз — евклидова',
    linestyle='--',
    linewidth=2
)

plt.title('прогноз 60 значений — евклидова метрика')
plt.xlabel('номер шага')
plt.ylabel('RCORR_E')
```

```
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

#график прогноза — манхэттенская метрика
plt.figure(figsize=(14, 6))

plt.plot(
    range(1, horizon + 1),
    actual_60,
    label='фактические значения',
    linewidth=2
)

plt.plot(
    range(1, horizon + 1),
    pred_manhattan,
    label='прогноз — манхэттенская',
    linestyle='--',
    linewidth=2
)

plt.title('прогноз 60 значений — манхэттенская метрика')
plt.xlabel('номер шага')
plt.ylabel('RCORR_E')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

#график прогноза — метрика Чебышева
plt.figure(figsize=(14, 6))

plt.plot(
    range(1, horizon + 1),
    actual_60,
    label='фактические значения',
    linewidth=2
)

plt.plot(
    range(1, horizon + 1),
    pred_chebyshev,
    label='прогноз — Чебышева',
    linestyle='--',
    linewidth=2
)

plt.title('прогноз 60 значений — метрика Чебышева')
plt.xlabel('номер шага')
plt.ylabel('RCORR_E')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

#расчёт MAE
mae_euclid = np.mean(
    np.abs(actual_60 - pred_euclid)
)

mae_manhattan = np.mean(
```

```
    np.abs(actual_60 - pred_manhattan)
)

mae_chebyshev = np.mean(
    np.abs(actual_60 - pred_chebyshev)
)


print()
print("MAE:")
print(f"евклидова: {mae_euclid:.6f}")
print(f"манхэттенская: {mae_manhattan:.6f}")
print(f"чебышева: {mae_chebyshev:.6f}")

#визуализация результата
plt.figure(figsize=(14, 6))

#последние 20 точек исторического ряда
tail = 20

plt.plot(
    range(N - tail, N),
    series[-tail:],
    label='обработанный ряд (последние 20)'
)


#координаты будущих 60 точек
future_x = np.arange(
    N,
    N + horizon
)


#фактические значения
plt.plot(
    future_x,
    actual_60,
    linewidth=2,
    label='фактические значения'
)


#прогноз евклидовой метрики
plt.plot(
    future_x,
    pred_euclid,
    linestyle='--',
    label='прогноз — евклидова'
)


#прогноз манхэттенской метрики
plt.plot(
    future_x,
    pred_manhattan,
    linestyle='--',
    label='прогноз — манхэттенская'
)


#прогноз Чебышева
plt.plot(
```

```
    future_x,
    pred_chebyshev,
    linestyle='--',
    label='прогноз — Чебышева'
)


#граница между известными и прогнозируемыми данными
plt.axvline(
    N - 1,
    linestyle='--',
    label='начало прогноза'
)


plt.title(
    'прогноз следующих 60 значений методом ближайших соседей'
)

plt.xlabel('номер отсчёта')
plt.ylabel('RCORR_E')

plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```